n = 83 K600 index lift: lift_scan in the root-index build, end-to-end cold

2026-10-19

n = 83 K600 index lift: lift_scan in the root-index build, end-to-end cold

Status (2026-10-19): protocol frozen; Tier 1 run 39290913632 started on commit `8d06c09d`; no result yet. The protocol (frozen 2026-10-19T17:02:53Z) and the schedule were committed before any Tier 1 timing. Every number below that is not quoted from an archived run comes from the cloud-agent VM; those are **local diagnostics**, not timing evidence (see `smoke/`).

Question

The n = 83 a = 1 K600 one-target index-calculus baseline is the rank-probe producer, `koblitz-n83-rankprobe-20261019` (PR #980, run 39214027289): end-to-end cold 166.0 s, `S` 6.39, gap to the tuned rho rT770 64.0 [53.7, 78.0]. Its cold path (setup + one-target online, 22.3 s) is 13% of that, and 88% of the cold path is the root-index build.

Canonicalizing the index build with lift_scan (`KIC_INDEX_CANON=lift`) instead of the selected canonicalizer has been a pre-registered secondary three times and never the primary, so it was never promoted:

run	arm	result (secondary, never decided)
state fold, 38819494806	cF / cFI	setup about 1.33x
rank cut, 38891404048	cRI	cold without rank 1.3377; end-to-end 1.3735 against the primary's 1.3336
rank probe, 39214027289	cPI	cold cP / cPI 1.3425; end-to-end 1.1424 against the primary's 1.1033; online 0.998

This experiment gives it its own frozen primary over the rank-probe producer: does it lower the **end-to-end cold** cost on new targets, with the stored states, the table layout, every relation, every probe count, every scalar and the factor-log table unchanged? It is an iteration under AGENTS.md "Success criterion"; a promoted result becomes the new n = 83 row. The expected end-to-end gain is about 3.5% (from run 39214027289's rank stage and cP / cPI cold means): an estimate, not a claim.

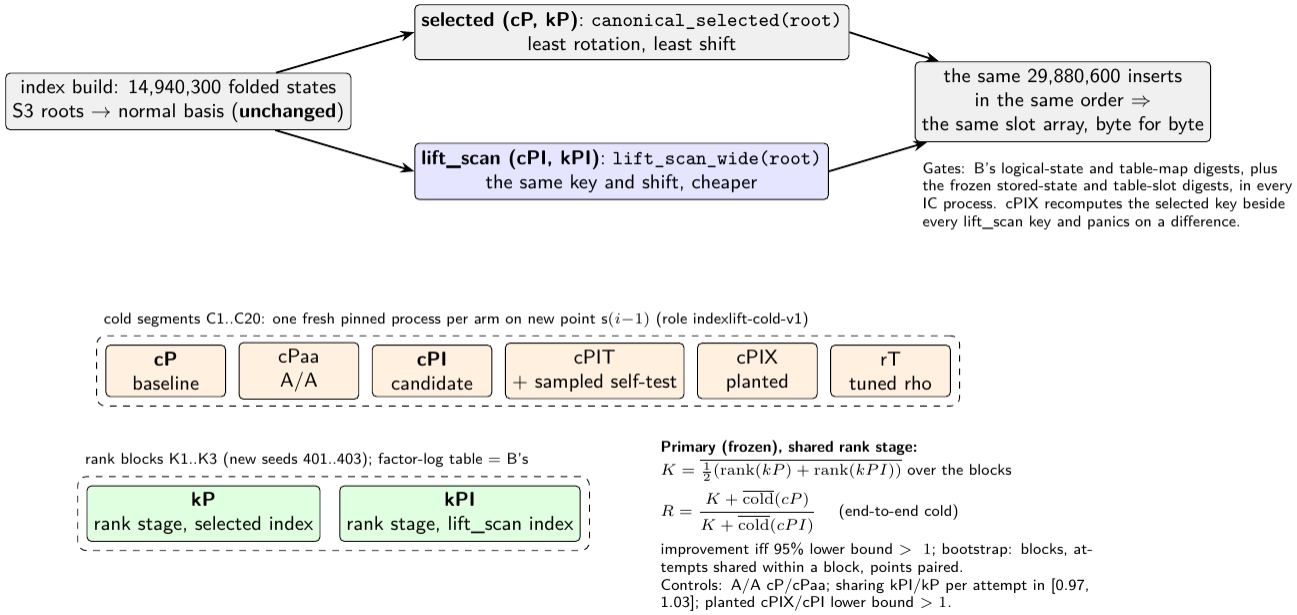


Figure 1: Design (from the frozen protocol; not a result). Source: figures/design.tex, vector: figures/design.pdf.

Exactness first: what the earlier runs show

Every IC process in those runs ran with `KIC_INDEX_DIGEST=1`, and `arm.py` gated it on B's index digests. Decoding the archived raw records of all three runs (`run_github_*/panel/runs/`):

run	lift-index processes	logical-state blake3	table-map sum / xor	entries	stored states	relation, probes, scalar per point
38819494806 (cFI)	16	`baf46d5a...` = B's	`be14a807...` / `8132bdcf...` = B's	29,878,182	14,940,300	= every other IC process
38891404048 (cRI)	20	= B's	= B's	29,878,182	14,940,300	= every other IC process
39214027289 (cPI)	20	= B's	= B's	29,878,182	14,940,300	= every other IC process

- **The stored states were the same.** The logical-state digest hashes every stored representative's bytes verbatim, in index order (and every mate rebuilt from one), so equal digests mean equal stored states.
- **The table map was the same** (an order-independent digest over every key and packed value), and the cold arms load B's verified factor-log table, so the table, relation, probes and scalar all agreed.
- **What was not checked:** the slot order of the root table. The map digest is order-independent, so a table holding the same map in other slots would have passed.

Why the layout is in fact identical, not only equivalent. `KIC_INDEX_CANON` changes one call in the index build: the canonical key and shift of each stored state's two normal-basis roots. The stored states are computed before and independently of that call. Both canonicalizers return the least rotation of the 83-bit word and the least shift that reaches it (83 is prime, so every word except 0 and all-ones has one such shift, and both return 0 for those two). So the table receives the same 29,880,600 inserts in the same order, and its slot array is the same word for word. Everything downstream reads only the stored states and the table.

The gate, defined precisely. Every IC process must reproduce:

1. B's logical-state blake3, table-map sum and xor, and 29,878,182 entries (as before);
2. two new layout digests, added by this producer after the target and outside every timed interval: `stored_states_blake3` (every stored state's bytes in stored order) and `table_slots_blake3` (every slot word, empty slots included, in slot order), equal to the frozen values in `protocol.json` (`online.layout_digest`);
3. per point, one relation, one probe count and one scalar across every IC process, the scalar replayed in the pinned oracle; per rank block, one relation digest, one per-attempt probe vector, one root-call total, one set of counters, and B's factor-log table, log for log.

The planted arm cPIX also recomputes the selected key and shift beside every lift_scan key on the runner and panics on any difference.

The change and the producer

`make_online_source.py` derives the producer in two steps:

1. `make_rankprobe_source.py` is the rank-probe experiment's `make_online_source.py`, verbatim, with its five verbatim parents. Its output must hash to the rank-probe protocol's frozen producer digest, `d4394df3...`; the derivation checks this.
2. Nine counted, exact replacements, none on a timed path of the baseline:
   - `KIC_INDEX_CANON=lift_plant`: the planted positive control (lift_scan keys, each checked against the selected canonicalizer's key and shift; it costs `lift` plus the work `lift` removes). `selected` and `lift` are the rank-probe producer's, unchanged.
   - `KIC_SCAN_SELFTEST=full|sampled` (default `full`, unchanged): `sampled` skips the Feistel scan order's exhaustive bijection sweep (2 keys x 2^26 positions) and keeps the folded index's sampled inverse check. For the secondary arm only.
   - the two layout digests (`KIC_INDEX_DIGEST=1`).
   - one test module: every index mode builds the same stored states and slot array on a 9-orbit base, and lift_scan equals the selected canonicalizer, key and shift, on 400,005 random and structured words.

The rank stage: shared, and why

Only the cold path changes. The rank stage runs identical code over an identical index (gated byte for byte above), so its cost is not a treatment effect. Measuring it separately per arm would add only the rank stage's process-to-process drift (about ±0.3% per process in run 39214027289, on 87% of the end-to-end cost) to a ratio whose treatment is entirely in the cold term; that drift is what pushed the earlier designs' simulated rejection rate at no effect to 0.08–0.125.

The rank stage is still measured in this run, for two reasons. The end-to-end cost needs it on this host: no number is imported from another run. And the candidate pipeline's own rank process, on a lift_scan-built index (kPI), must give B's factor-log table with the baseline's relations and probes. So each rank block runs kP (selected index) and kPI (lift_scan index), and the primary uses the **shared** rank stage

```
K = mean over blocks of (rank(kP) + rank(kPI)) / 2,
E2E(c) = K + mean cold(c),
```

with the same K in the numerator and the denominator. The pooling is checked on the run by the sharing control (kPI / kP per attempt inside [0.97, 1.03]). The per-arm form, (kP + cP) / (kPI + cPI), is a secondary.

Frozen Tier 1 design

One GitHub `ubuntu-24.04` runner, one pinned CPU, Tier 1 counters, one fresh process per run. Every IC arm runs the index-lift producer (`ic_online_indexlift_r770`) with the 16-byte compact table over the folded index, lift_scan target keys, the Feistel order and the prefetch pipeline.

Arm	Kind	Index build	Role
`cP`	cold	selected	**baseline** cold: the rank-probe producer's cold path
`cPaa`	cold	selected	A/A of cP
`cPI`	cold	lift_scan	**candidate** cold
`cPIT`	cold	lift_scan, sampled scan self-test	pre-registered secondary
`cPIX`	cold	lift_plant	planted positive control
`rT`	rho	—	tuned rho rT770 (Fold/64/DP6), seed 890000 + point seed
`kP`	rank	selected	shared rank stage (the rank-probe producer's rank process, `frame`)
`kPI`	rank	lift_scan	shared rank stage (the candidate's rank process)

- **Cold panel.** 20 new public points (`inputs/cold_targets.json`, role `indexlift-cold-v1`, built by `make_targets.py`), checked disjoint from every earlier n = 83 panel of the law, all regenerated: Q0 and B's panel, scanorder-heldout, combined-primary-v1, probepipe-primary-v1, statefold-cold-v1, tablefilter-primary-v1, stacked-cold-v1, rankcut-cold-v1, **rankprobe-cold-v1**, and the 64 `eval` points of n131-ref-walk-20261015. No scalar is known for any of them.
- **Rank blocks.** K1–K3 use the new rank seeds 401–403 (1,800 new rank targets), each one process of kP and of kPI.
- **Segment order.** O1 (cP, cPI, cPIT and cPIX on Q0, exactness only), C1–C6, K1, C7–C12, K2, C13–C18, K3, C19–C20, with new shuffle seeds. Time guard 300 minutes; job limit 350.

Primary (pre-registered). R = E2E(cP) / E2E(cPI) with the shared rank stage. Bootstrap, 10,000 draws, seed 20261119: the blocks are resampled; within each drawn block the attempt indices are resampled and used for both rank processes; the points are resampled, paired across cP and cPI. **Improvement** iff the 95% lower bound of R exceeds 1; regression iff the upper bound is below 1; otherwise not separated. **Invalid** if any gate or control fails or any scheduled row is missing.

Controls.

- A/A: E2E(cP) / E2E(cPaa) and cold cP / cPaa inside [0.97, 1.03].
- Sharing: rank stage per attempt kPI / kP inside [0.97, 1.03].
- Planted: cold cPIX / cPI with its lower bound above 1.

Gates. As defined under "Exactness first", plus: every row valid; the arm's index canonicalizer and scan self-test mode recorded; cPIX checks exactly 29,880,600 roots and every other process none; Q0 gives the archived Feistel identity (1,987,852 probes) in all four Q0 arms; every rank process reaches full rank with `KIC_RANK_SOLVE=frame`, one frame per attempt, and B's table.

Secondaries (none decides): (kP + cP) / (kPI + cPI); the pre-registered cPIT, E2E(cP) / E2E(cPIT) and cold cPI / cPIT; cold, setup, index build, online, peak RSS and scan self-test cP / cPI; rank kPI / kP; `S` per arm and the gaps to rT770.

Why cPIT (a setup-side secondary)

The archived setup profile of run 39214027289 (`make_setup_profile.py`, `inputs/setup_profile_archived.json`; Tier 1, not this experiment's timing) splits the median cold process:

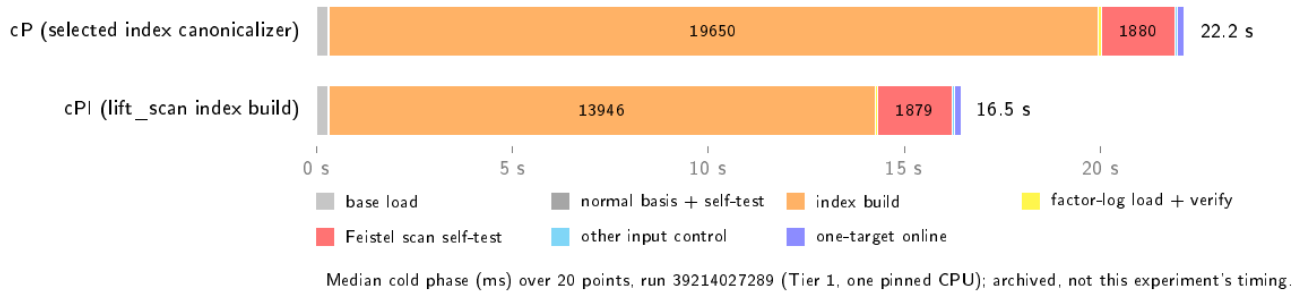


Figure 2: Median cold phases of the archived cP and cPI processes (source figures/profile.tex, vector figures/profile.pdf).

After the index build, the largest setup piece is the Feistel scan order's exhaustive bijection self-test, 1.88 s in every cold process (8.5% of cP's cold path, 11.4% of cPI's). It checks that the Feistel permutation is a bijection on [0, 29,880,000), for two fixed keys that are not the key the scan uses. That is a property of the construction: it does not depend on the target, the base or the scan key, and the crate tests check it exhaustively on the real state count (the workflow runs them on the frozen source). cPIT skips the sweep and keeps the sampled inverse check. Expected about 1.9 s, about 1% end to end. It is a secondary, never decides, and is reported separately: removing a check is engineering only if the check is redundant, and a reviewer may weigh that differently.

Power (`power_check.py`, `analysis/power.json`)

The prior is `inputs/power_prior.json`, built by `make_power_prior.py` from run 39214027289: its kP / kPaa rank attempts (four blocks; two processes of the same rank code, as kP and kPI are here) and its cR / cP cold times (two processes of the same cold code, so the null is a real A/A pair). A simulated candidate divides the cold times by the cold effect that gives the stated end-to-end ratio at the prior's means (rank stage 143.7 s, cold 22.3 s). 200 simulations per cell, 300 bootstrap draws each, the frozen shared-rank bootstrap.

end-to-end effect	cold effect	2 blocks	3 blocks	4 blocks
1.000 (null)	1.000	0.030	0.025	0.020
1.002	1.015	1.00	1.00	1.00
1.005	1.039	1.00	1.00	1.00
1.010 (minimal effect of interest)	1.080	1.00	1.00	1.00
1.020	1.171	1.00	1.00	1.00
1.035 (expected)	1.337	1.00	1.00	1.00

- The rejection rate at no effect is at the nominal 0.025 (Monte Carlo standard error about 0.011): with the rank stage shared, the rank-stage process drift that inflated the earlier designs' rate to 0.08–0.125 is not in the ratio.
- Two blocks already have power 1.00 at the minimal effect; the design runs three (the frozen minimum), because K enters every `S` and gap to rho directly.
- Expected (not an input): (K + cP) / (K + cPI) = 1.0355 from run 39214027289's means.

Local exactness and diagnostics (cloud-agent VM)

See `smoke/README.md`. In short:

- **Tests.** All 20 producer tests pass, two of them new (every index mode builds the same stored states and slot array; lift_scan equals the selected canonicalizer, key and shift, on 400,005 words), plus 4 tuned-rho and 24 lift2 tests.
- **Every index mode on Q0** (selected, lift, lift with the sampled self-test, lift_plant): one stored-state digest `d8cfe443...` and one table-slot digest `5f53764e...`, B's digests, and the archived Q0 relation, scalar and 1,987,852 probes. lift_plant compared all 29,880,600 index roots with no difference. **The layout is identical**, so the gate is byte identity of the layout, not only equivalence.

- **Full K600 rank stages** (seed 301, point s000 of the rank-probe panel, the inputs of the archived block K1): kP and kPI both reproduce the archived kP row exactly (relation digest, all 600 per-attempt probe counts, root calls, counters, target relation, scalar and probes) and B's table. Rank stage kPI / kP 0.996 on this VM (diagnostic only).
- **Runner, wrapper and verifier** on O1, C1 and K1: all 12 rows valid and every correctness check passes.
- **Verifier on synthetic rows** (run 39214027289's rows renamed): with the archived cPI as candidate, R = 1.0354 [1.0337, 1.0372], "improvement"; with a copy of the baseline, 1.0000 [0.9998, 1.0002], "not separated".

Files

File	Role
`protocol.json`, `make_protocol.py`, `schedule.json`, `make_schedule.py`	frozen protocol, its builder, frozen run order
`make_online_source.py`, `make_rankprobe_source.py`, `make_rankcut_source.py`, `make_stacked_source.py`, `make_tablefilter_source.py`, `make_probepipe_source.py`, `make_combined_source.py`, `online/`, `lift2/`	the index-lift producer (the rank-probe derivation verbatim, digest-checked, plus `lift_plant`, `KIC_SCAN_SELFTEST` and the layout digests); tuned rho; the two-limb canonicalizer crate
`restore.py`, `check_log_table.py`	restore and digest-check crypto 770f6a69 and every local source; re-verify B's factor-log table
`make_targets.py`, `inputs/cold_targets.json`	the new cold panel
`inputs/archived_identities.json`	archived Q0 identity
`make_power_prior.py`, `inputs/power_prior.json`, `power_check.py`, `analysis/power.json`	power prior (run 39214027289), simulation, result
`make_setup_profile.py`, `inputs/setup_profile_archived.json`, `figures/profile.*`	archived setup profile and its graph
`arm.py`, `run_schedule.py`	one arm per pinned process; the schedule runner
`verify_receipt.py`	gates, statistics, decision rule; `result.json`, `result_table.md`
`figures/design.*`	design diagram (TikZ source, PDF, PNG)
`make_figures.py`	result figure, once the run is archived
`smoke/`	local diagnostics: exactness of every index mode on Q0 and of full rank stages, tests, verifier check
`README.pdf`	this report: `pandocREADME.md-oREADME.pdf--pdf-engine=pdflatex-Hreport_header.tex--lua-filterreport_filter.lua-Vgeometry:margin=2cm-Vfontsize=10pt-Vcolorlinks=true`